MPLAB Harmony 6.25 – Multicast Buffer Logic

Compiled conversation notes, scenarios, and integration code

1. Original requirement

The screen has Radio and Ethernet interface checkboxes, a manual-entry text field, an Add button, a Multicast button, address-book checkboxes, a Done button, and a label that displays the multicast buffer. Manual input is first held in **Input_buffer**. Clicking Add transfers it into the final multicast buffer. Multicast checkbox selections remain temporary until Done is clicked. The final buffer can contain at most **5 entries**, separated only by the pipe character (**|**).

2. Required ordering behaviour

• **Manual first:** Add manual ID 1234, then select 45665 and 1236 in multicast and click Done. Result: `1234|45665|1236`.

• **Multicast first:** Select 1234, 45665 and 1236 and click Done, then manually add 565. Result: `1234|45665|1236|565`.

• **Alternating additions:** Every successful Add or Done appends to the same active buffer. Do not clear the existing buffer when opening multicast.

• **Capacity:** remaining slots = 5 – current entry count. With 0 existing entries allow 5 selections; with 1 allow 4; with 2 allow 3; with 3 allow 2; with 4 allow 1; with 5 allow 0.

• **Validation:** reject empty values, duplicates, values containing '|', and any addition that would exceed 5 entries. Avoid leading or trailing pipe characters.

• **Interface separation:** keep Radio and Ethernet data in separate buffers/counts so IDs and IP addresses are not mixed.

3. Suggested shared variables

```
#define MAX_ENTRIES 5
#define BUFFER_SIZE 170

char Input_buffer[32] = {0};
char multicast_buffer[BUFFER_SIZE] = {0};

int multicast_count = 0;

char selected_data[MAX_ENTRIES][32];
bool selected[MAX_ENTRIES] = {false};
```

4. Shared append function

Both the Add button and Done button must call this same function. It appends a separator only when the buffer already contains data.

```
bool AppendToMulticastBuffer(const char *data)
{
    if (data == NULL || data[0] == '\\0')
        return false;

    if (multicast_count >= MAX_ENTRIES)
        return false;

    if (strchr(data, '|') != NULL)
        return false;

    /* Check duplicate entries */
    char temp[BUFFER_SIZE];
    strcpy(temp, multicast_buffer);

    char *token = strtok(temp, "|");

    while (token != NULL)
    {
        if (strcmp(token, data) == 0)
```

```
            return false;

        token = strtok(NULL, "|");
    }

    /* Append separator only when buffer is not empty */
    size_t len = strlen(multicast_buffer);

    if (len > 0)
        strcat(multicast_buffer, "|");

    strcat(multicast_buffer, data);

    multicast_count++;

    return true;
}
```

5. Add button callback

The text-field event should copy the entered value into Input_buffer. The Add callback then transfers it into the shared buffer and clears Input_buffer only after success.

```
void AddButton_Callback(void)
{
    if (Input_buffer[0] == '\\0')
        return;

    if (AppendToMulticastBuffer(Input_buffer))
    {
        Input_buffer[0] = '\\0';

        /* Clear your textfield widget here. */
        /* Refresh your multicast display label here. */
    }
}
```

6. Done button callback

Checkbox selections are temporary. When Done is clicked, append each checked item to the same multicast buffer. Uncheck an item only after it has been successfully appended.

```
void DoneButton_Callback(void)
{
    int i;

    for (i = 0; i < MAX_ENTRIES; i++)
    {
        if (selected[i])
        {
            if (multicast_count >= MAX_ENTRIES)
                break;

            if (AppendToMulticastBuffer(selected_data[i]))
            {
                selected[i] = false;
            }
        }
    }

    /* Refresh your multicast display label here. */
}
```

7. Checkbox capacity logic

```
int remaining = MAX_ENTRIES - multicast_count;

if (checked)
{
    int selected_count = 0;

    for (int i = 0; i < MAX_ENTRIES; i++)
    {
        if (selected[i])
            selected_count++;
    }
```

```
    if (selected_count >= remaining)
    {
        /* Reject additional checkbox selection. */
        return;
    }

    selected[index] = true;
}
else
{
    selected[index] = false;
}
```

For a real address book with more than five candidates, size the temporary checkbox array to the number of address-book candidates, not MAX_ENTRIES. The selected-count check should count only newly selected entries that are not already present in the final buffer.

8. Expected outputs

```
Manual 1234, then multicast 45665 and 1236:
1234|45665|1236

Multicast 1234, 45665 and 1236, then manual 565:
1234|45665|1236|565

Maximum five entries:
1234|45665|1236|565|2333
```

9. MPLAB integration notes

• Call the Add callback from the Add button's Legato event handler.

• Read the text-field string into Input_buffer before calling AddButton_Callback().

• When a multicast checkbox changes, update the corresponding selected[] state and enforce the remaining capacity.

• On Done, call DoneButton_Callback().

• After either callback, update the label using multicast_buffer directly. Do not add another pipe in the label update code.

• Do not reset multicast_buffer or multicast_count when opening the multicast selection screen.

• If Radio and Ethernet must remain independent, maintain separate buffers and counts for each interface, and make the append function operate on the currently selected interface.

10. Important C string note

In C source code, string literals are written with double quotes, for example `"|"`. Those quote characters are not part of the stored data. The actual buffer contents should look like `1234|45665|1236` with no extra quotes, spaces, or backslashes.